

Pangu Ultra: 在升腾 NPU 上推动稠密大语言模型极限

Pangu Team, Huawei

PanguTech@huawei.com

摘要

我们提出 Pangu Ultra, 一种拥有 1350 亿参数的稠密 Transformer 大语言模型 (LLM), 在升腾 Neural Processing Units (NPUs) 上完成训练。尽管近年来 LLM 规模和能力持续快速提升, 但训练如此大规模模型仍面临显著的优化与系统挑战。为稳定训练过程, 我们提出了 depth-scaled sandwich normalization, 有效消除了深层模型训练中的 loss spike。我们在 13.2 万亿高质量、多样化 token 上进行预训练, 并在后训练阶段进一步增强模型推理能力。为高效执行这一大规模训练, 我们使用 8,192 张升腾 NPU, 并配套一系列系统优化。多项公开基准评测表明, Pangu Ultra 在能力上显著推进了稠密 LLM 的 SOTA, 相比 Llama 405B 和 Mistral Large 2 等模型更具竞争力, 且与参数规模更大的稀疏模型 DeepSeek-R1 相比也取得了接近结果。我们的探索表明, 升腾 NPU 具备高效训练百亿级以上稠密模型的能力。模型与系统将面向商业客户开放。

1 引言

大语言模型 (LLM) 正在重塑人工智能的技术版图与行业认知。其出色能力正推动越来越多 AI 应用落地, 并带来广泛商业机会。由此, 业界普遍围绕 scaling law 展开竞争, 持续推动参数规模增长。尽管 Transformer [68] 是大模型的主流架构, 但在超大规模场景下选择稀疏还是稠密仍存争议。参数规模超过千亿后, 以 Mixture of Experts (MoE) 为代表的稀疏架构 (如 DeepSeek [46, 19]) 展现出接近人类的语言与思维能力 [36], 这使稀疏模型成为冲击 LLM 上限的热门路径。

与此同时, Qwen [11, 72]、Llama [25]、Gemma [67] 等稠密模型系列, 在 100B 以下参数区间依然广受采用, 原因在于其在特定能力上的强表现与部署友好性。稠密模型参数通常更易优化, 而稀疏模型中的动态组件在稳定训练时往往需要额外启发式策略。此外, 稠密结构在推理时参数使用是确定性的, 系统优化也更直接。基于此, 我们进一步探索大规模稠密模型的潜力, 验证其在多类任务上可与 SOTA MoE 模型竞争。

对于稠密模型而言, 参数规模与网络深度是释放模型潜力的两个关键维度。参数规模对性能至关重要并构成 scaling law 核心 [38]; 近期研究 [73, 50] 则指出深度对推理能力也有显著影响。然而, 在这两个维度同时推进会带来明显挑战。更深模型通常更易出现训练不稳定, 表现为训练损失曲线中的尖峰。实验观察显示, 这些尖峰可能将模型推离理想参数区域, 并对后续训练造成难以恢复的伤害。与此同时, 在稠密模型中训练千亿级参数需要协同数千个 AI 处理器, 也带来系统效率层面的严峻挑战。

在本工作中, 我们提出 Pangu Ultra: 一种包含 1350 亿参数、94 层的稠密 Transformer 架构。该模型规模处于当前高性能稠密模型前沿 [11, 72, 25, 67]。针对深层训练挑战, 我们推测 loss spike 源于梯度波动, 进而阻碍收敛并可能导致训练发散。因此, 我们提出两项技术: depth-scaled sandwich norm 与 tiny initialization, 二者都旨在维持稳定梯度范数。具体而言, 我们将 pre-layer norm [47] 替换为 sandwich norm [20], 并按模型深度对 post-layer normalization 的初始化参数进行缩放, 从而有效控制梯度波动范围。除此之外, 我们按模型宽度与深度缩放权重初始化标准差, 形成 tiny initialization。两项技术共同提升了训练全过程的梯度稳定性, 消除了 Pangu Ultra 训练中的 loss spike, 并提升了整体性能。

在实践中, 我们在 13.2 万亿 token 自建语料上对 Pangu Ultra 进行预训练。预训练阶段包含三种不同数据配方的语料阶段, 设计目标依次为: 先建立知识与语言能力, 再直接强化推理能力, 最后进一步提升主动推理能力。模型上下文窗口由 4K 逐步扩展到 128K。在后训练阶段, 我们先通过高效监督微调 (SFT) 进行冷启动, 使用精心构建的指令数据; 随后再通过强化学习 (RL) 进一步优化。Pangu Ultra 的完整训练在上述流程中保持稳定。

为支持百亿级以上参数的大规模训练，我们使用由 8,192 张升腾 NPU 组成的集群，并引入一系列系统优化提升效率。首要挑战是在大规模下降低 pipeline bubble [29]，其根因与 batch size 约束有关 [35]。我们在升腾集群上同时使用四类并行：数据并行 (DP)、张量并行 (TP) [63]、序列并行 (SP) [39] 和流水并行 (PP) [30, 51]。随着集群扩大，每个 DP 分配到的 mini-batch 变小，pipeline bubble 比例会上升。为缓解该问题，我们使用额外的虚拟流水并行 (VPP) 调度 [52] 并做细粒度调优，将 PP bubble 比例从 30.45% 降至 6.8%。第二个挑战是长序列下维持高训练效率。attention mask 生成与自注意力计算在长上下文都非常耗时且占内存。我们使用面向升腾 NPU 的 NPU Fusion Attention (NFA) 算子 [4, 18, 17]，其支持 reset attention mask 场景，并在调用 NFA 前无需显式构造 attention mask，从而提升计算效率并降低内存成本。通过多项细粒度系统优化，我们在 8,192 张升腾 NPU 上训练 Pangu Ultra 时实现了超过 50% 的模型 FLOPs 利用率 (MFU) [14]。

在公开评测基准上，Pangu Ultra 在几乎所有主要语言任务上超过 Llama 405B、Mistral Large 2 123B 等稠密 LLM，并与总参数规模超过 500B 的稀疏模型取得有竞争力的结果。这些结果说明稠密模型能力仍有较大探索空间。Pangu Ultra 也进一步证明了升腾 NPU 适合用于探索大规模稠密语言模型的能力边界。

2 模型架构

Pangu Ultra 的基础架构与 Llama 3 [25] 类似。模型包含 1350 亿参数，隐藏维度为 12,288，采用 SwiGLU [60] 前馈网络 (FFN)，中间层维度为 28,672，并堆叠 94 层。Pangu Ultra 在注意力模块中使用 Group Query Attention (GQA)，配置 96 个 query heads 和 8 个 KV heads，以减少 KV-cache 开销。

为解决大规模稠密 LLM 在训练稳定性与收敛性上的核心挑战，我们引入了两个关键改动：以 Depth-Scaled Sandwich-Norm 替代原有归一化结构，并采用 TinyInit 参数初始化。结合这两项技术后，Pangu Ultra 相较此前稠密模型取得了显著提升。

2.1 Depth-Scaled Sandwich-Norm

大规模稠密模型通常更依赖增加网络深度 [22]，而 MoE 模型通常通过增大宽度扩展规模 [19]。但深度增加会带来更高的训练不稳定风险。在预训练代价极高的前提下，大规模稠密 LLM 的稳定训练尤为关键。已有研究发现，Pre-Layer Normalization (Pre-LN) 可提升深层 Transformer 的反向传播效率 [69]，因此被广泛用于 Transformer 架构的大语言模型 [22, 11, 19]。

然而，在 pre-LN 结构中，各子层输出尺度的波动容易导致训练不稳定 [66]。为缓解该问题，sandwich-norm [20] 在残差连接前对每个子层输出再施加一层归一化。尽管该方法可以保持单层输出尺度稳定，但跨多层残差连接不断累积输出范数，仍可能引发训练不稳定。

为此，我们提出 depth-scaled sandwich norm：将 sandwich norm 与深度缩放初始化结合。归一化层通过可学习的 gamma 参数调节逐层输出幅度，并将其初始值按网络深度的倒数比例缩放。图 1 展示了 depth-scaled sandwich-norm 与 pre-norm 架构的差异。其形式如下：

$$\begin{aligned} \mathbf{h} &\leftarrow \mathbf{h} + \text{Norm}(\gamma_{\text{attn}}, \text{ATTN}(\text{Norm}(\mathbf{h}))), & \gamma_{\text{attn}} &= \frac{c_{\text{attn}}}{\sqrt{L}}, \\ \mathbf{h} &\leftarrow \mathbf{h} + \text{Norm}(\gamma_{\text{mlp}}, \text{MLP}(\text{Norm}(\mathbf{h}))), & \gamma_{\text{mlp}} &= \frac{c_{\text{mlp}}}{\sqrt{L}}, \end{aligned} \quad (1)$$

其中， L 表示层数， c_{attn} 与 c_{mlp} 分别为注意力层与前馈网络层输出标准差的初始值。对于 Pangu Ultra，我们设置 $c_{\text{attn}} = 0.283$ 、 $c_{\text{mlp}} = 0.432$ 。

2.2 模型初始化

已有研究 [53] 指出，模型初始化对训练稳定性和最终性能至关重要。基于 Transformer 的 LLM 通常采用小尺度初始化 [53]，即将权重按标准差 $\sqrt{\frac{2}{5d}}$ 的正态分布初始化，其中 d 为隐藏维度。另一个常见做法是按 $1/\sqrt{L}$ [57] 缩放残差层权重初始化，其中 L 为层数。

我们的发现是：同时结合模型深度与宽度进行缩放，即使用 $\sqrt{\frac{1}{2dL}}$ ，可以带来更快的 loss 收敛速度以及更好的下游任务表现。我们将该初始化称为 TinyInit。我们推测 TinyInit 让模型各处参数尺度更一致，从而更利于优化与收敛。

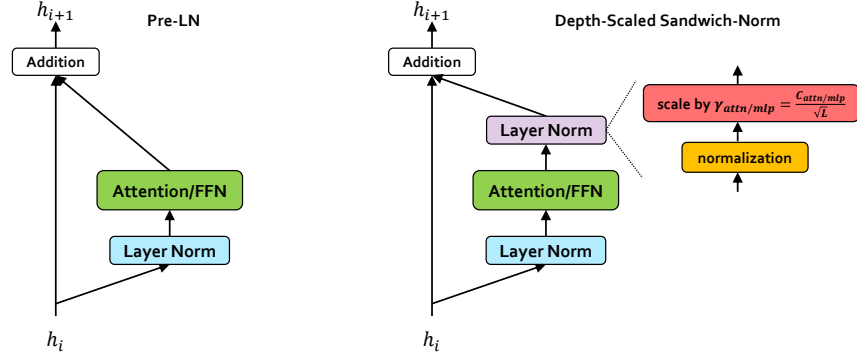


Figure 1: Pre-Layer Norm (Pre-LN) 与 Depth-Scaled Sandwich-Norm (DSSN) 结构对比。DSSN 在注意力和 FFN 模块前后都施加归一化层，而 Pre-LN 仅使用一层归一化。DSSN 还引入了深度缩放初始化，这是原始 sandwich norm 不具备的。

研究 [66] 还表明，Embedding 层应采用与其他层不同的初始化策略。具体来说，将 embedding 权重标准差保持在接近 1 的范围可能有助于训练稳定。我们的实验表明，将其初始化为 0.5 可取得较好的模型性能。

2.3 Tokenizer

Tokenizer 的设计会显著影响模型表现。理想词表需要同时兼顾领域覆盖（支持文本、数学、代码等多类任务）与编码效率（用更少 token 表示数据）。常见方法如 Byte-Pair Encoding (BPE) [62] 和 SentencePiece [40]，通常直接在全量训练数据上统计词频构建词表。但这种方式容易出现领域失衡：通用文本等高频领域主导词表，而数学、代码等数据量较小的专业领域表示不足。

Pangu Ultra 采用面向领域的词表策略。我们分别在通用中文、通用英文、代码和数学等多个领域独立做词频分析，构建领域词表后再合并去重，得到包含 153,376 个唯一 token 的统一词表。在保持整体压缩效率的同时，该方法实现了更均衡的跨领域表示。表 1 给出了各领域 token 分布。

Table 1: Pangu Ultra 统一词表中的 token 分布。

领域	Token 数量	占比 (%)
English	68,017	44.35
Chinese	41,053	26.77
Other	30,573	19.93
Latin-based languages	4,507	2.94
Arabic	2,755	1.80
Korean	2,733	1.78
Mathematics	2,139	1.39
Japanese	1,599	1.04
Total	153,376	100.00

3 模型训练

本节介绍我们的训练流水线，其整体思路与当前领先语言模型（如 DeepSeek-V3 [19] 与 Llama 3 [22]）相似。训练过程包含三个主要阶段：预训练、长上下文扩展和后训练。每个阶段均配套特定的训练策略与数据构建方法，以逐步提升模型能力。

3.1 预训练阶段

我们首先介绍 Pangu Ultra 预训练的数据构建方案及其质量验证方法，然后阐述长上下文扩展的实践方式，最后给出预训练超参数细节。

3.1.1 数据构建

Pangu Ultra 的预训练语料由 tokenizer (见第 2.3 节) 产生, 共计 13.2T 高质量、多样化 token。表 2 展示了预训练的三阶段流程: *general* 阶段、*reasoning* 阶段与 *annealing* 阶段。这三个阶段分别用于逐步建立通用知识与语言能力、强化推理能力, 并进一步精炼知识与行为表现。各阶段数据量分别为: 12T (其中由两个子阶段构成, 分别为 7.4T 与 4.6T)、0.8T 与 0.4T token。

Table 2: Pangu Ultra 预训练数据配比。

数据集	General	Reasoning	Annealing
General English	54%	14%	21%
General Chinese	13%	6%	20%
Multi-lingual	8%	4%	3%
Instruction	2%	11%	20%
Math	6%	28%	18%
Code	17%	37%	18%

在初始的 *general* 阶段, 我们使用以通用语言能力和常识知识构建为目标的语料。该阶段主要包含英文与中文数据, 来源覆盖网页、图书、百科等多种渠道, 同时也纳入多语种与工业领域数据。基于第 3.1.2 节的数据质量评估结果, 我们在第二个子阶段相较第一个子阶段使用了更高质量的数据。

在第二个 *reasoning* 阶段, 我们显著提高高质量数学与代码数据的占比, 使其超过语料总量的 60%, 以强化 Pangu Ultra 的推理能力。代码数据包含纯代码样本与文码混合样本; 数学数据也包含大量中英文文本。此外, 我们还大规模引入由 LLM 生成的合成数据, 以进一步丰富语料分布。

第三个 *annealing* 阶段用于帮助模型巩固并高效应用前两个阶段获得的知识和推理能力。因此我们进一步提升 *instruction* 数据占比, 约占语料的 20%。我们构建了覆盖广泛主题的内部题库, 并生成短链与长链推理 (CoT) 答案。这些推理路径经过精细化打磨, 以保证表达清晰和逻辑连贯。

总体来看, Pangu Ultra 预训练数据经过精心设计, 兼顾高质量、多样性和低冗余。我们为数据打上质量与难度标签, 并在三个阶段中对推理数据采用课程式采样策略, 使训练样本从简单逐步过渡到复杂。

3.1.2 数据质量评估

数据质量评估是提升整体数据质量的关键环节。在 Pangu Ultra 训练中, 我们结合基于规则的启发式方法与基于模型的评估方法进行质量提升。

在基于模型的质量评估中, 我们使用 Pangu 系列模型作为基础评估器。为使质量评估更贴近人工价值判断, 我们基于人工标注数据集对评估器进行微调。微调后的评估器应用于超过 10T token 的大规模预训练语料, 对样本在清洁度、流畅度、教育价值和信息丰富度等多个维度打分, 并据此进行优先级采样: 高质量样本获得更高采样概率。

为验证数据质量评估的有效性, 我们使用 26 亿参数代理模型开展消融实验。实证结果显示: 在达到相近性能时, 使用低评分数据训练所需 token 数量是高分高质量数据的 1.6 倍。因此, 高质量数据对提升训练效率至关重要。

3.1.3 预训练参数

Pangu Ultra 采用 AdamW 优化器 [48], weight decay 设为 0.1, epsilon 设为 1×10^{-8} 。动量参数设为 $\beta_1 = 0.9$ 、 $\beta_2 = 0.95$, 梯度裁剪阈值设为 1.0。为提升训练稳定性和整体性能, Pangu Ultra 的预训练按照以下阶段进行:

0T–7.4T tokens 序列长度设为 4K (RoPE base = 1×10^4)。batch size 从 1,024 增至 1,536 (在 1.2T 时) 和 2,048 (在 1.9T 时)。增大 batch size 可提升训练效率与吞吐。学习率采用余弦衰减, 从 1×10^{-4} 降至 1×10^{-5} , 并设置 4,000 个 warmup steps 以保证早期训练稳定。

7.4T–12.0T tokens 序列长度保持 4K, batch size 保持 2,048。该阶段学习率固定为 1×10^{-5} 。

12.0T–12.8T tokens 序列长度提升到 8K (RoPE base = 1×10^5), batch size 降为 1,536。学习率采用余弦调度, 从 1×10^{-5} 衰减到 7.5×10^{-6} 。

3.2 长上下文扩展

LLM 对长上下文输入的理解能力对于长链思考与实际应用都至关重要。在预训练后期，我们通过长序列数据训练使 Pangu Ultra 支持最大 128K 上下文窗口。训练分两步渐进进行：先扩展到 32K，再进一步扩展到 128K。

Rotary Position Embedding (RoPE) [64] 是支持超长输入序列的核心模块。现有开源 LLM 通常通过两类方法扩展上下文长度：一是增大 RoPE base frequency [64, 32]，二是采用 YaRN 等方法 [55, 22, 19]。我们的实验发现，只要超参数设置合理，这两类方法表现相近；Pangu Ultra 最终采用增大 base frequency 的方案。为确定长上下文扩展时 RoPE 的 base frequency，我们在目标序列长度下评估不同 base frequency 的“Needle In A Haystack” (NIAH) 离线表现，选择效果最佳者，以获得较低的长上下文训练初始 loss。在实践中，32K 阶段选择 1.6×10^6 ，128K 阶段选择 2.56×10^7 。Pangu Ultra 长上下文训练超参数如下：

8K 到 32K 阶段序列长度扩展至 32K (RoPE base = 1.6×10^6)，batch size 为 384，学习率为 7.5×10^{-6} ，与前一阶段末端学习率保持一致。

32K 到 128K 阶段序列长度进一步扩展到 128K (RoPE base = 2.56×10^7)，batch size 降至 96，学习率保持 7.5×10^{-6} 。

3.3 后训练对齐

在后训练阶段，Pangu Ultra 通过监督微调 (SFT) 与强化学习 (RL) 对齐人类偏好。该阶段重点在于构建高质量、多样化 instruction 数据，并设计可扩展且高效的训练策略。

3.3.1 后训练数据

在后训练数据构建中，我们重点关注数据质量、多样性和复杂度。数据池覆盖广泛领域与任务类型，包括通用问答、AIGC、文本分类与分析、编程、数学、逻辑推理与工具调用等。这些任务覆盖金融、医疗与公共服务等应用场景。数据来源包括开源 instruction 数据集、真实工业查询以及由预训练语料派生的合成问题。

为提升多样性，我们依据熵规律 [74]，沿两个正交维度进行采样：领域与任务类型。我们使用不同粒度的分层标签模型来支持均衡采样。数据质量通过规则校验与模型校验联合控制，以过滤低质量或歧义样本。

为更好激发 Pangu Ultra 的推理能力，后训练数据中约六分之五为推理类任务（数学、代码、逻辑）。数据覆盖不同复杂度区间，并重点强调中高难度样本。

3.3.2 后训练策略

在后训练阶段，我们首先通过 SFT 让 Pangu Ultra 建立基础的指令跟随能力。随后采用基于结果反馈的 RL，进一步提升 Pangu Ultra 在推理、对齐和指令遵循方面的能力。

我们实现了一个面向 Ascend 基础设施优化、具备时延容忍能力的强化学习框架，详细内容将在后续报告中给出。该框架支持在 Ascend 上进行高效大规模策略优化。为引导 RL 过程，我们构建了混合奖励系统，为数学、代码和通用问题求解提供任务特定反馈。该系统结合确定性奖励信号与基于模型的评估，从而实现稳定且高效的策略优化。

4 训练系统

在 13.2 万亿 token 上训练 135B 参数的 Pangu Ultra，需要在大规模计算集群中同时保障训练稳定性与效率。本节从两个关键维度展开：并行化策略与系统级优化技术，分别见第 4.2 节和第 4.3 节。总体上，我们在 8,192 张 Ascend NPU 上训练 Pangu Ultra 时，实现了超过 52% 的模型 FLOPs 利用率 (MFU)。

4.1 计算集群配置

我们部署了由 8,192 张 Ascend Neural Processing Units (NPUs) [5, 6] 组成的集群用于训练 Pangu Ultra。集群中每个节点包含 8 张 NPU，节点内通过 Huawei Cache Coherence System (HCCS) 全互连拓扑连接，每张设备配备 64GB 内存。节点间通信通过 RoCE 网络实现，跨节点 NPU 间带宽为 200 Gbps。

4.2 面向模型扩展的并行策略

为实现模型训练扩展¹，我们结合多种并行方式将模型分布到多张 NPU 上，包括数据并行 (DP) [43]、张量并行 (TP) [63]、序列并行 (SP) [39] 与流水并行 (PP) [30, 51]。在 Pangu Ultra 中，我们采用 128 路 DP 并结合 ZERO [58]，以降低模型参数及优化器状态的内存成本。采用 8 路 TP 利用节点内高带宽高效传输激活；采用 8 路 PP 利用节点间连接，因为其仅需在分区边界传输激活。但如已有研究指出 [35, 30, 51, 56]，随着训练集群规模扩大，PP 会因 batch size 约束遭遇严重 pipeline bubble [35]。对于 one-forward-one-backward (1F1B) 调度，bubble ratio 定义为 $\frac{p-1}{p-1+n}$ ，其中 p 为流水阶段数， n 为每个 DP 的 micro-batch 数。该比例对应加速器空转时间，如图 2 所示。大规模集群会增加 DP 数量，由于 batch size 限制，每个 DP 分到的 micro-batch 减少，bubble ratio 随之上升。因此，降低 bubble ratio 是提升系统效率的关键。在这一背景下，我们采用每设备 6 路虚拟 PP 阶段的 interleaved pipeline-parallel 调度 [52]，将 bubble ratio 从 30.45% 降至 6.8%。通过对 PP 与 VPP 阶段负载均衡进行细致调参，我们在 8,192 NPU 集群上得到约 43% MFU 的基线性能。

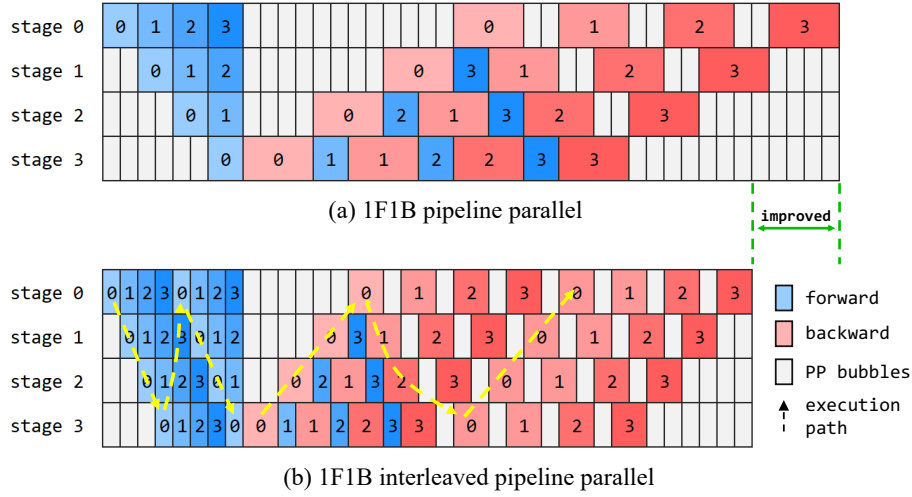


Figure 2: 流水并行与交错式流水并行调度示意。

4.3 系统优化

在第 4.2 节 43% MFU 基线之上，我们进一步加入系统级优化以持续提升训练效率。通过融合算子、基于子序列切分的上下文并行、数据缓存与共享机制等优化，Pangu Ultra 训练效率得到显著提升。这些优化使系统 MFU 超过 52%，相较于第 4.2 节基线实现约 9% 相对提升。

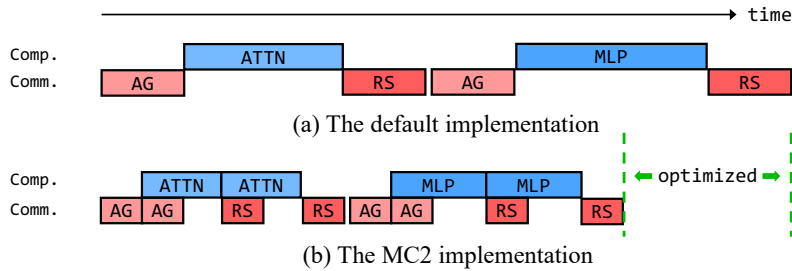


Figure 3: 默认 Transformer 计算流程与 MC2 方法对比。实际训练中，MC2 会将通信与计算任务融合到单一 kernel。

¹Pangu Ultra 训练基于 MindSpeed [8] 与 Megatron [7, 63] 框架，二者提供了完整并行策略与系统优化能力。

4.3.1 Kernel 融合

Kernel 融合是 LLM 训练中常用的效率优化技术。它将多个操作合并到单个 kernel 中，以减少对全局内存的访问次数 [17]。在 Pangu Ultra 训练中，我们对关键算子进行融合，显著提升了硬件利用率和整体训练效率。

MC2 - 计算与通信融合 (Merged Compute and Communication) 张量并行结合序列并行时，需要通过 All-Gather (AG) 与 Reduce-Scatter (RS) 在分布式设备间交换输入/输出激活。该过程使矩阵乘 (MatMul) 与 AG/RS 通信存在强依赖关系，从根本上限制了 TP 通信与计算流程的重叠空间。我们实现了 MC2 [2, 3] 来解决这一问题，通过将 MatMul 与通信操作融合，缓解该瓶颈。MC2 将大计算和大通信任务拆分为细粒度子任务，并采用流水化执行以最大化通信与计算重叠。因此，MC2 显著降低了通信时延并提升硬件利用率 (见图 3)。

NPU Fusion Attention 长序列训练中，自注意力的时间和内存开销会随序列长度增长呈二次增加。为解决这一问题，Flash Attention (FA) 因高性能已成为 LLM 训练标准技术之一 [18, 17]。Pangu Ultra 采用专为 Ascend NPU 优化的自注意力融合算子 NPU Fusion Attention (NFA) [9]，可在多类自注意力计算场景下带来系统级收益。

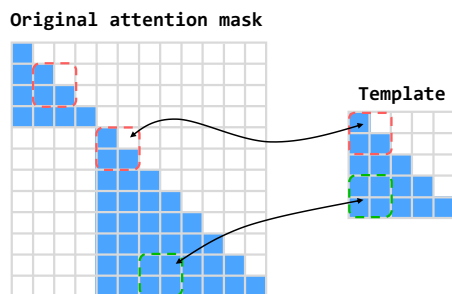


Figure 4: NFA 算子中的 attention mask 压缩示例。

值得一提的是，Pangu Ultra 使用 reset attention mask 策略，以阻止同一序列内不同文档间发生自注意力。这要求为每个序列计算对应 attention mask，带来明显内存和计算开销。为降低生成 attention mask 的时间与内存成本，NFA 引入了 mask 压缩优化。如图 4 所示，NFA 使用 2048×2048 因果 mask 作为模板，在融合注意力算子内构造实际计算 mask。每次迭代时，Pangu Ultra 根据文档结束 token (eod) 位置获取真实序列长度，并将其作为输入提供给 NFA，以加速自注意力计算。NFA 的详细使用方式可参见 Ascend 文档 [9]。

其他融合优化除 MC2 与 NPU 优化融合注意力外，我们还在 RMSNorm [77]、SwiGLU [60]、RoPE [64] 等关键模块，以及梯度累积、PP 发送/接收通信等关键流程中引入了系列 kernel 融合优化。这些融合算子旨在降低 kernel 启动与内存访问开销，同时保持较高数值精度并提升整体训练性能。

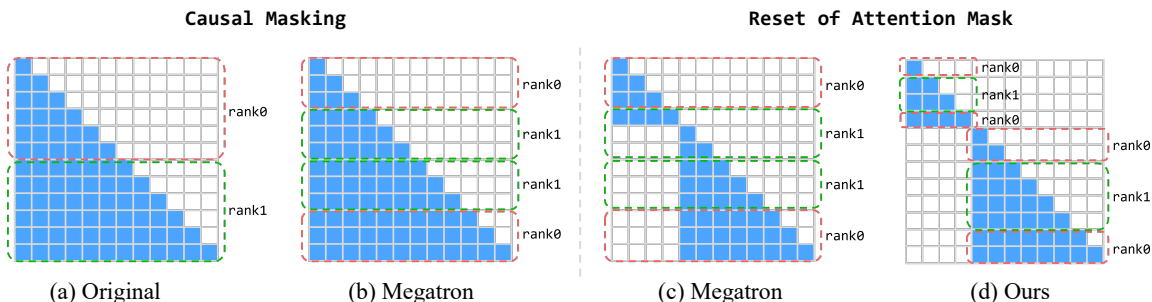


Figure 5: 用于上下文并行的子序列切分机制示例。

4.3.2 长上下文训练优化

长上下文能力对长文档摘要、对话式 AI 等场景越来越重要。但长序列训练在时间和内存复杂度上都面临挑战。为提升长上下文训练效率，我们提出两项关键策略。

用于上下文并行的子序列切分上下文并行 (CP) 是训练超长序列的关键方法之一, 通过切分输入序列来降低内存消耗 [44, 33]。但在因果 mask 下, 若直接将序列切为 CP 份, 会导致 Ring Self-Attention (RSA) [44] 负载严重不均衡 (图 5(a))。Megatron-LM 通过将序列切分为 $2 \times CP$ 份来缓解该问题, 每个 rank 同时接收顶部和底部的 chunk, 从而在 CP 组内平衡负载 (图 5(b)) [7]。但在 reset attention mask 场景下, 该方法依然存在负载不均衡 (图 5(c))。因此, 在 128K 长上下文训练中, 我们提出了更均衡的 CP 切分策略: 每个 rank 在每个子序列内负责计算两个 chunk (图 5(d))。

快速 mask 生成与数据复用 当 Pangu Ultra 训练序列长度扩展到 128K 时, attention mask 生成或真实序列长度计算仍会带来不可忽视的性能开销。另外, 在 reset attention mask 的训练场景中, 每个 VPP 阶段每次迭代都需获取对应 mask 或真实序列长度, 造成重复计算与额外开销。我们通过两项优化解决该问题: (1) 使用高效 NPU 算子计算 attention mask, 替代 CPU 侧构造方式, 从而加速 mask 生成并消除 CPU 与 NPU 间数据传输; (2) 启用跨 VPP 阶段的 mask 共享机制, 由第一阶段 (VPP0) 生成 attention mask, 并在同一 rank 的不同 VPP 阶段共享, 避免重复计算和额外内存开销。

5 实验结果

本节汇报 Pangu Ultra 的评测结果, 包括预训练阶段表现与后训练结果。此外, 我们给出较完整的消融研究, 用于分析模型架构设计并讨论 Pangu Ultra 训练中的关键观察。

5.1 预训练损失曲线

图 6 展示了 Pangu Ultra 在整个预训练期间的损失曲线。损失曲线中的每一段对应第 3.1.3 节定义的一个训练阶段。各阶段都呈现稳定下降趋势。第二阶段虽然因学习率恒定而下降速度放缓, 但相关性指标仍持续提升。

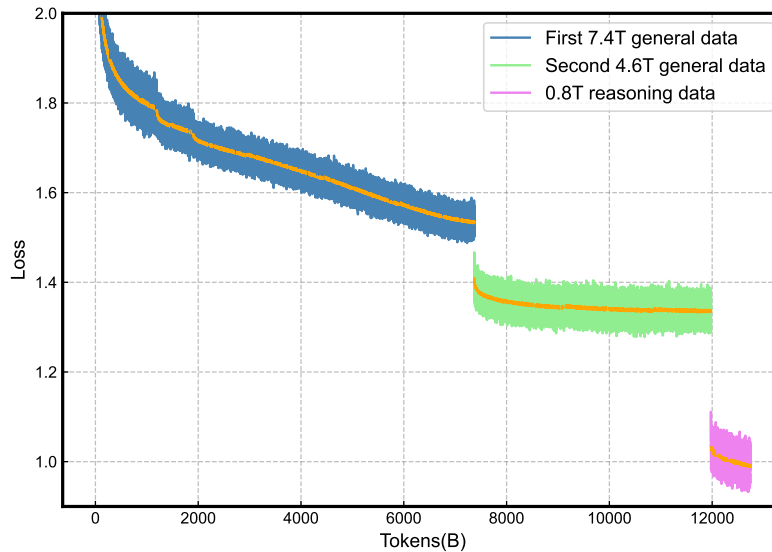


Figure 6: Pangu Ultra 在预训练阶段的训练损失曲线。

Zero loss spike 如图 6 所示, 整个预训练过程没有出现 loss spike。尽管此类尖峰在 LLM 训练中并不少见 [66], 这里的“零尖峰”结果进一步说明了 depth-scaled sandwich norm 与 TinyInit 对训练稳定性的关键作用。loss spike 对模型性能的负面影响将在第 5.4.1 节进一步讨论。

5.2 预训练阶段

Benchmarks 我们在多个公开基准上评估 Pangu Ultra 基座模型, 覆盖语言理解、问答、代码生成与数学求解等领域。评测以英文与中文数据集为主, 并补充少量多语种基准以增强覆盖面。

- 语言理解：使用 *Hellaswag* [76]、*Winogrande* 评估语境推理；使用 *DROP* [21]、*RACE* [42]、*ARC* [15] 系列评估阅读理解；并结合 *PIQA* [12]、*Natural Questions* [41]、*TriviaQA* [37] 评估知识检索能力。
- 问答：包括中文知识基准 *C-Eval* [31]，英文领域知识基准 *MMLU* [27] 及其进阶版本 *MMLU-Pro* [70]，并使用 *BigBenchHard* [65] 评估复杂问题求解能力。
- 代码生成与理解：采用 *HumanEval* [13] 与 *MBPP* [10] 评估标准代码生成任务，并使用 *CruXEval* [26] 评估代码理解与推理。
- 数学推理：使用 *CMath* [71]、*GSM8K* [16] 评估基础算术与简单问题，*MATH* [28] 评估高阶数学推理，*MGSM* [61] 评估多语种数学求解能力。

Baselines & Comparison Settings 我们将 Pangu Ultra 与多个强基线比较，覆盖稠密模型 (Qwen2.5-72B、Llama-405B) 和 MoE 模型 (DeepSeek-V3)。对于基座模型，大多数评测采用 few-shot，少数任务使用 zero-shot。多数基准通过与标准答案精确匹配打分，代码生成任务则采用执行验证。

Evaluation Results 表 3 对比了 Pangu Ultra 预训练基座模型与其他主流模型。总体看，Pangu Ultra 在大多数英文通用基准和全部中文基准上达到最优。在代码与数学任务上虽略低于 DeepSeek V3，但整体表现仍具竞争力。

进一步分析可见，Pangu Ultra 在中文基准上优势明显，不仅超过 Qwen 2.5 72B，也优于当前最强中文模型之一 DeepSeek V3。此外，与 Llama 3.1 405B 相比，Pangu Ultra 在多数高难度基准上得分更高，而训练 FLOPs 仅约为 Llama 405B 的 29%。这说明我们的模型架构设计与训练数据质量是有效的。

5.3 后训练与推理能力

Benchmarks 我们从推理与非推理两个维度，对 Pangu Ultra 的能力进行综合评测：

- 复杂推理任务包含三类子任务：数学能力 (*AIME 2024* [49]、*MATH-500*)、代码竞赛 (*LiveCodeBench* [34]) 以及科学推理 (*GPQA Diamond* [59])；
- 通用语言理解与推理能力，使用 *MMLU-Pro* [24] 与 *Arena Hard* [45]。

Baselines & Comparison Settings 我们将 Pangu Ultra 与强基线模型比较，包括 GPT-4o-0513、推理模型 DeepSeek-R1、Hunyuan-T1，以及大规模稠密模型 Qwen2.5-72B-Instruct 与 Mistral-Large 2。评测指标采用多次独立运行后的平均 Pass@1。

Evaluation Results 表 4 展示了 Pangu Ultra 与其他模型的对比结果。Pangu Ultra 在 AIME 2024、MATH-500、GPQA、LiveCodeBench 等推理基准上达到领先，同时保持较强的通用语言理解能力。

与稠密 LLM (Qwen、Mistral-Large 2) 相比，Pangu Ultra 在推理任务上的优势尤其明显。该优势来自预训练中 0.8T 推理数据 (见第 3.1.3 节) 的支撑，推理增强后的 base 模型也显著助力了后训练阶段表现。

5.4 消融研究

本节补充模型架构消融实验，并分析训练过程中的关键行为，以便更深入理解稠密 LLM 的训练特性。

5.4.1 Depth-scaled Sandwich-norm

我们开展实验验证 depth-scaled sandwich norm 相比 pre-norm 架构的有效性。实验使用 130 亿参数稠密 Transformer，在相同超参数下训练 3000 亿 token，对两种配置进行对比。

图 7 显示，depth-scaled sandwich-norm 可以稳定梯度范数并有效消除 loss spike，从而加快训练收敛。我们在两个组合基准上评估性能：EN basic (多个英文基准组合) 和 ZH basic (中文基准组合)。此外，还使用 LAMBADA [54] (英文) 和 WPLC [23] (中文) next-token 预测任务进行补充验证。结果清楚表明：在预训练阶段抑制 loss spike 对模型性能至关重要。

Table 3: Pangu Ultra 与代表性模型在语言、代码与数学能力基准上的对比。粗体表示该行最佳结果，下划线表示 Pangu Ultra 在稠密模型中的最佳结果。

	Benchmark (Metric)	# Shots	Qwen2.5 72B Base	Llama-3.1 405B Base	DeepSeek V3 Base	Pangu Ultra Base
	Architecture	-	Dense	Dense	MoE	Dense
	# Activated Params	-	72B	405B	37B	135B
	# Total Params	-	72B	405B	671B	135B
English	BBH (EM)	3-shot	79.8	82.9	87.5	79.1
	MMLU (EM)	5-shot	85.0	84.4	87.1	85.4
	MMLU-Pro (EM)	5-shot	58.3	52.8	64.4	<u>63.1</u>
	DROP (F1)	3-shot	80.6	86.0	89.0	61.0
	ARC-Easy (EM)	25-shot	98.4	98.4	98.9	100.0
	ARC-Challenge (EM)	25-shot	94.5	95.3	95.3	97.0
	HellaSwag (EM)	10-shot	84.8	89.2	88.9	99.0
	PIQA (EM)	0-shot	82.6	85.9	84.7	98.0
	WinoGrande (EM)	5-shot	82.3	85.2	84.9	91.0
	RACE-Middle (EM)	5-shot	68.1	74.2	67.1	97.0
	RACE-High (EM)	5-shot	50.3	56.8	51.3	97.0
	TriviaQA (EM)	5-shot	71.9	82.7	82.9	90.5
	NaturalQuestions (EM)	5-shot	33.2	41.5	40.0	52.7
	AGIEval (EM)	0-shot	75.8	60.6	79.6	80.4
Code	HumanEval (Pass@1)	0-shot	53.0	54.9	65.2	81.1
	MBPP (Pass@1)	3-shot	72.6	68.4	75.4	72
	CRUXEval-I (EM)	2-shot	59.1	58.5	67.3	<u>61.8</u>
	CRUXEval-O (EM)	2-shot	59.9	59.9	69.8	<u>61.5</u>
Math	GSM8K (EM)	8-shot	88.3	83.5	89.3	89.3
	MATH (EM)	4-shot	54.4	49.0	61.6	62.5
	MGSM (EM)	8-shot	76.2	69.9	79.8	75.1
	CMath (EM)	3-shot	84.5	77.3	90.7	78.2
Chinese	CLUEWSC (EM)	5-shot	82.5	83.0	82.7	95.0
	C-Eval (EM)	5-shot	89.2	72.5	90.1	90.3
	CMMLU (EM)	5-shot	89.5	73.7	88.8	91.7
	CMRC (EM)	1-shot	75.8	76.0	76.3	86.0
	C3 (EM)	0-shot	76.7	79.7	78.6	99.0
	CCPM (EM)	0-shot	88.5	78.6	92.0	93.0

Table 4: Pangu Ultra 与代表性模型在各基准上的比较。† 表示结果来自 Artificial Analysis [1]。

Model	AIME 2024	MATH-500	GPQA Diamond	LiveCode Bench	ArenaHard	MMLU-pro
GPT-4o-0513	9.3	74.6	49.9	32.9	80.4	72.6
Qwen2.5-72B	16.0	83.1	49	27.6	81.2	72.0
Mistral-Large 2†	11.0	73.6	48.6	29.3	-	69.7
Hunyuan-T1	79.8	96.2	69.3	64.9	91.9	87.2
DeepSeek-R1	79.8	97.3	71.5	65.9	92.3	84.0
Pangu Ultra	80.8	97.4	74.2	66.5	91.5	84.4

为进一步验证初始化中深度缩放因子的作用，我们将不含 Eq. (1) 中 $\sqrt{L}$ 缩放的原始 sandwich-norm 与我们方法进行对比。实验采用 16 亿参数、94 层代理模型（与 Pangu Ultra 深度一致），用于检验 sandwich-norm 在超深 Transformer 中的效果。如图 8 所示，原始 sandwich-norm 仍会出现一定 loss spike，而 depth-scaled sandwich-norm 训练更平稳且达到更低 loss。

5.4.2 Tiny Initialization

我们进一步验证第 2.2 节提出的 TinyInit 的有效性。在训练到 1020 亿 token 后，采用 TinyInit（标准差 $\sqrt{\frac{1}{2dL}}$ ）初始化的 Pangu Ultra，明显优于传统初始化基线（标准差分别为 $\sqrt{\frac{2}{5d}}$ 与 $\sqrt{\frac{2}{5dL}}$ ）。结

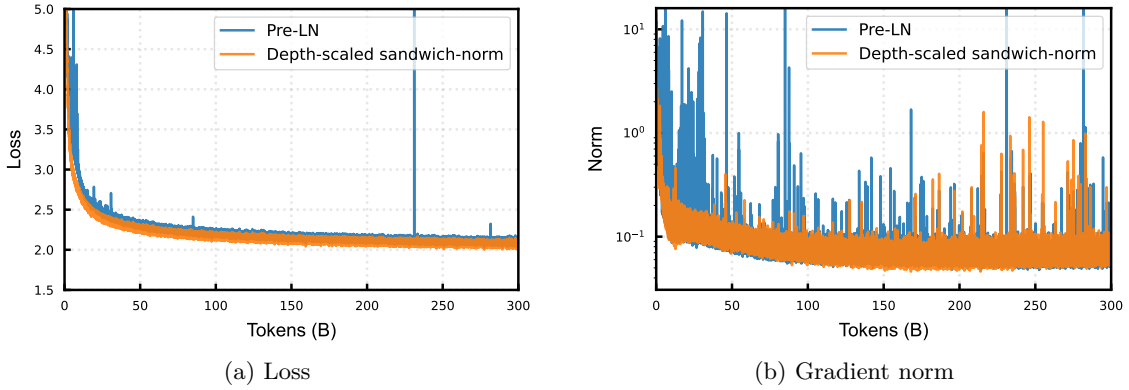


Figure 7: 13B 模型在 Pre-LN 与 Depth-Scaled Sandwich-Norm (DSSN) 下的预训练 loss 与梯度范数对比。Pre-LN 曲线存在明显尖峰并损害模型，而 DSSN 曲线更平滑。

Table 5: Pre-LN 与 Depth-scaled Sandwich-Norm 的性能比较。

Model	Tokens (B)	EN basic	ZH basic	LAMBADA	WPLC
Pre-LN	300	0.42	0.52	0.675	0.194
Depth-scaled sandwich-norm	300	0.45	0.54	0.693	0.224

果见表 6。其中 BIG-bench (aug) 为我们内部基于原始 BIG-bench 做数据增强后的测试集，用于降低测试集泄漏影响。

5.4.3 Pangu Ultra 的层统计特性

稳定的激活尺度 图 9 展示了 Transformer 各层 attention 与 FFN 模块的激活模式，包括均值、标准差和 top-1 激活值。激活分布整体稳定：在保持清晰分层模式的同时，标准差在整个预训练过程中维持一致尺度。我们观察到“super activations”现象：浅层中其幅值可达 10^3 量级，这与 Llama 模型中的观察一致 [75]。值得注意的是，图 9 显示 top-1 激活值会随层深增加逐步下降，意味着其对最终输出的相对影响变小。

Depth-scaled sandwich norm 的逐层模式 图 10 展示了各 sandwich-norm 层缩放参数 γ 的分布，呈现以下关键现象：四个 LayerNorm 的 γ 参数在训练过程中均值/标准差均下降，与 weight decay 效应一致；post-norm 的 γ 呈明显层依赖模式：其标准差随层深增加而显著上升；pre-norm

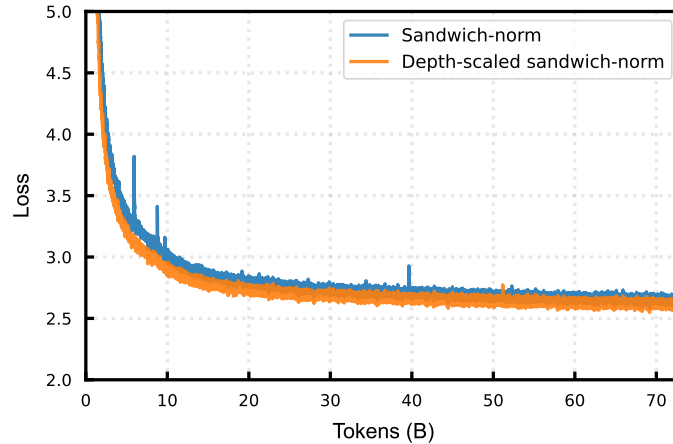


Figure 8: 94 层 1.6B 模型在原始与 depth-scaled sandwich-norm 下的预训练 loss 对比。原始 sandwich-norm 在训练中仍会出现 loss spike。

Table 6: 传统初始化与 TinyInit 的性能比较。

Model	Tokens (B)	EN basic	ZH basic	LAMBADA	WPLC	C-Eval	MMLU	BIG-bench (aug)
Baseline	102	0.444	0.538	0.694	0.229	0.476	0.473	0.357
TinyInit	102	0.456	0.537	0.727	0.257	0.524	0.502	0.384

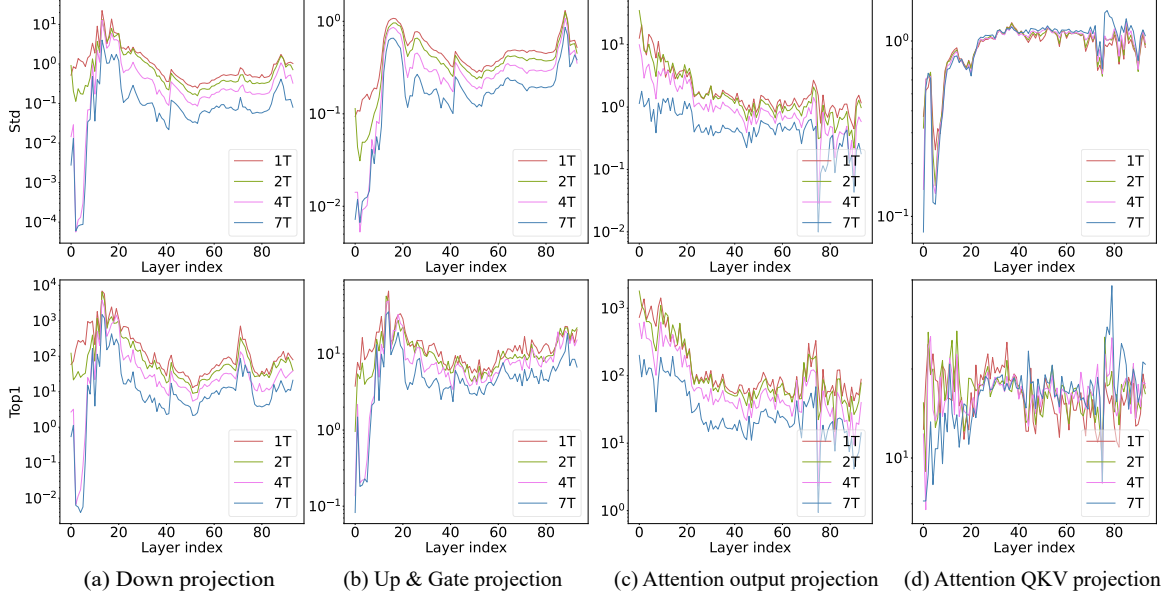


Figure 9: attention 与 FFN 模块激活统计, 包括均值、标准差和 top-1 值。每条曲线对应 1T、2T、4T、7T 不同训练 token 位置。”Std” 行显示跨层激活尺度稳定; ”Top 1” 行显示 down projection 与 attention output projection 中存在”super activations”, 其量级处于合理范围, 并与 LLaMA 中观察到的现象相当 [75]。

的 γ 在层间保持相对恒定的标准差。该模式揭示了一个有趣行为: 浅层主要依赖残差连接, 而深层随着缩放参数 γ 幅值增大, 逐步更依赖 Transformer 子层输出。

6 结论

我们提出 Pangu Ultra: 一个在升腾 NPU 上训练、拥有 1350 亿参数的稠密语言基础模型。为应对大规模深层模型训练挑战, 我们提出 depth-scaled sandwich-norm, 使 Pangu Ultra 在无明显 loss spike 的前提下获得了优异训练稳定性。在 8,192 张升腾 NPU 上完成 13.2 万亿 token 预训练与长上下文扩展后, 模型又通过监督微调与强化学习进一步提升推理能力。大量实验结果表明, Pangu Ultra 不仅超过了 Llama 405B、Mistral Large 2 等 SOTA 稠密模型, 也在与 DeepSeek-R1 等更大规模稀疏模型的对比中表现出竞争力。这说明我们提出的架构与系统优化方案有效, 为未来可扩展、高效率 LLM 训练提供了可行路径。此外, 我们的实践经验也表明, 升腾 NPU 具备训练百亿级乃至更大规模稠密模型的能力。

References

- [1] Artificial analysis. <https://artificialanalysis.ai/>.
- [2] Ascend mc2. <https://gitee.com/qingfenxiaochong/MindSpeed/blob/master/docs/features/mc2.md>.
- [3] Ascend mc2. <https://www.hiascend.com/developer/techArticles/20240613-1>.
- [4] Flash attention. <https://github.com/Dao-AI-Lab/flash-attention>.
- [5] Huawei atlas 800t a2. <https://e.huawei.com/cn/products/computing/ascend/atlas-800t-a2>.

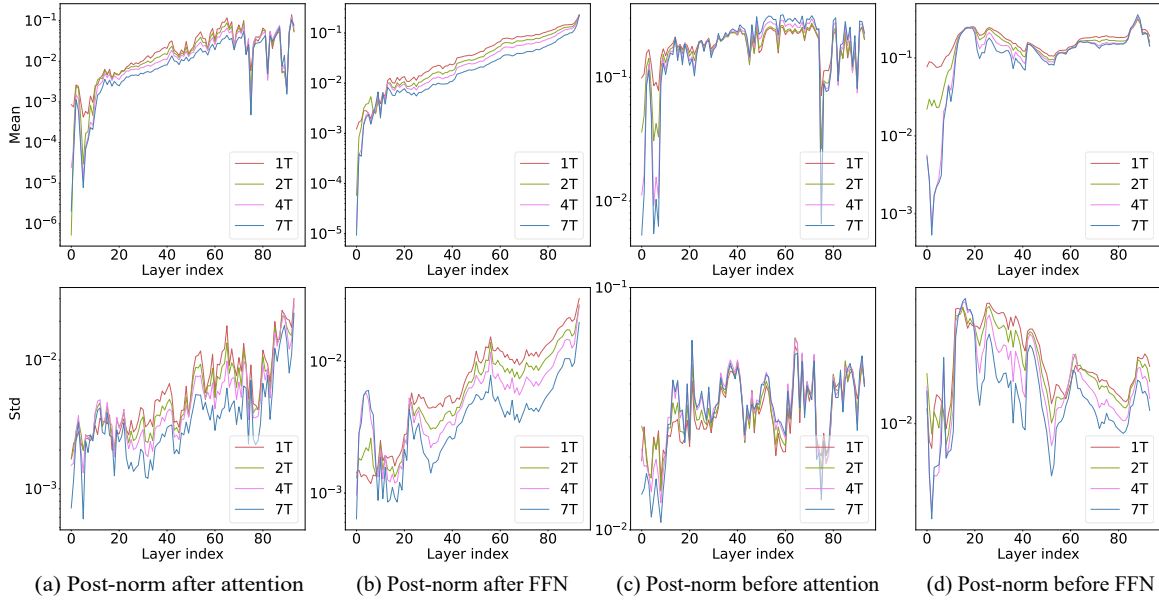


Figure 10: sandwich-norm 中 γ 参数分布 (均值与标准差)。每条曲线对应 1T、2T、4T、7T 不同训练 token 位置。两个 post-norm 呈现清晰逐层模式： γ 的均值和标准差随深度增加而上升。更大的 post-norm γ 表明深层更强调 Transformer 输出而非残差连接。

- [6] Huawei atlas 800t a2 technical specifications. <https://support.huawei.com/enterprise/en/doc/ED0C1100349804/2bf2c017/technical-specifications?idPath=23710424|251366513|22892968|252309113|254184887>.
- [7] Megatron-lm. <https://github.com/NVIDIA/Megatron-LM>.
- [8] Mindspeed. <https://gitee.com/ascend/MindSpeed>.
- [9] Npu fusion attention. https://www.hiascend.com/document/detail/zh/Pytorch/60RC1/apiref/apilist/ptaoplist_000139.html.
- [10] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. Program synthesis with large language models. *ArXiv*, abs/2108.07732, 2021.
- [11] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- [12] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language. In *AAAI Conference on Artificial Intelligence*, 2019.
- [13] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé, Jared Kaplan, Harrison Edwards, Yura Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mo Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, David W. Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William H. Guss, Alex Nichol, Igor Babuschkin, Suchir Balaji, Shantanu Jain, Andrew Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew M. Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *ArXiv*, abs/2107.03374, 2021.
- [14] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm

- Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omer-nick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways, 2022.
- [15] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *ArXiv*, abs/1803.05457, 2018.
- [16] Karl Cobbe, Vineet Kosaraju, Mo Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *ArXiv*, abs/2110.14168, 2021.
- [17] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2024.
- [18] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.
- [19] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Cheng-gang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojuan Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shut-ing Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanbiao Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. Deepseek-v3 technical report, 2025.
- [20] Ming Ding, Zhuoyi Yang, Wenyi Hong, Wendi Zheng, Chang Zhou, Da Yin, Junyang Lin, Xu Zou, Zhou Shao, Hongxia Yang, and Jie Tang. Cogview: Mastering text-to-image generation via transformers. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 19822–19835. Curran Associates, Inc., 2021.
- [21] Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *North American Chapter of the Association for Computational Linguistics*, 2019.

- [22] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [23] Huibin Ge, Chenxi Sun, Deyi Xiong, and Qun Liu. Chinese wplc: A chinese dataset for evaluating pretrained language models on word prediction given long-range context. In *Conference on Empirical Methods in Natural Language Processing*, 2021.
- [24] Aryo Pradipta Gema, Joshua Ong Jun Leang, Giwon Hong, Alessio Devoto, Alberto Carlo Maria Mancino, Rohit Saxena, Xuanli He, Yu Zhao, Xiaotang Du, Mohammad Reza Ghasemi Madani, et al. Are we done with mmlu? *arXiv preprint arXiv:2406.04127*, 2024.
- [25] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [26] Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida Wang. Cruxeval: A benchmark for code reasoning, understanding and execution. *ArXiv*, abs/2401.03065, 2024.
- [27] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Xiaodong Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *ArXiv*, abs/2009.03300, 2020.
- [28] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Xiaodong Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *ArXiv*, abs/2103.03874, 2021.
- [29] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.
- [30] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Xu Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. Gpipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 103–112, 2019.
- [31] Yuzhen Huang, Yuzhuo Bai, Zhihao Zhu, Junlei Zhang, Jinghan Zhang, Tangjun Su, Junteng Liu, Chuancheng Lv, Yikai Zhang, Jiayi Lei, Fanchao Qi, Yao Fu, Maosong Sun, and Junxian He. C-eval: A multi-level multi-discipline chinese evaluation suite for foundation models. *ArXiv*, abs/2305.08322, 2023.
- [32] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-coder technical report, 2024.
- [33] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models, 2023.
- [34] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- [35] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. Megascale: Scaling large language model training to more than 10,000 gpus, 2024.
- [36] Cameron R Jones and Benjamin K Bergen. Large language models pass the turing test. *arXiv preprint arXiv:2503.23674*, 2025.
- [37] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *ArXiv*, abs/1705.03551, 2017.

- [38] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [39] Vijay Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models, 2022.
- [40] Taku Kudo and John Richardson. SentencePiece: A simple and language independent sub-word tokenizer and detokenizer for neural text processing. In Eduardo Blanco and Wei Lu, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, Brussels, Belgium, November 2018. Association for Computational Linguistics.
- [41] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur P. Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc V. Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- [42] Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard H. Hovy. Race: Large-scale reading comprehension dataset from examinations. *ArXiv*, abs/1704.04683, 2017.
- [43] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. Pytorch distributed: Experiences on accelerating data parallel training. *CoRR*, abs/2006.15704, 2020.
- [44] Shenggui Li, Fuzhao Xue, Chaitanya Baranwal, Yongbin Li, and Yang You. Sequence parallelism: Long sequence training from system perspective. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2391–2404, Toronto, Canada, July 2023. Association for Computational Linguistics.
- [45] Tianle Li, Wei-Lin Chiang, Evan Frick, Lisa Dunlap, Banghua Zhu, Joseph E. Gonzalez, and Ion Stoica. From live data to high-quality benchmarks: The arena-hard pipeline, April 2024.
- [46] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.
- [47] Liyuan Liu, Xiaodong Liu, Jianfeng Gao, Weizhu Chen, and Jiawei Han. Understanding the difficulty of training transformers. In *EMNLP (1)*, pages 5747–5763. Association for Computational Linguistics, 2020.
- [48] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [49] MAA. Codeforces. American Invitational Mathematics Examination - AIME 2024, 2024. <https://maa.org/math-competitions/american-invitational-mathematics-examination-aime>.
- [50] William Merrill and Ashish Sabharwal. A little depth goes a long way: The expressive power of log-depth transformers, 2025.
- [51] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. Pipedream: generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP 2019, Huntsville, ON, Canada, October 27-30, 2019*, pages 1–15. ACM, 2019.
- [52] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '21, New York, NY, USA, 2021*. Association for Computing Machinery.
- [53] Toan Q Nguyen and Julian Salazar. Transformers without tears: Improving the normalization of self-attention. *arXiv preprint arXiv:1910.05895*, 2019.

- [54] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and R. Fernández. The lambada dataset: Word prediction requiring a broad discourse context. *ArXiv*, abs/1606.06031, 2016.
- [55] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models, 2023.
- [56] Penghui Qi, Xinyi Wan, Guangxing Huang, and Min Lin. Zero bubble pipeline parallelism, 2023.
- [57] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.
- [58] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020*, page 20. IEEE/ACM, 2020.
- [59] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024.
- [60] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- [61] Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, Dipanjan Das, and Jason Wei. Language models are multilingual chain-of-thought reasoners. *ArXiv*, abs/2210.03057, 2022.
- [62] Yusuke Shibata, Takuya Kida, Shuichi Fukamachi, Masayuki Takeda, Ayumi Shinohara, Takeshi Shinohara, and Setsuo Arikawa. Byte pair encoding: A text compression scheme that accelerates pattern matching. 1999.
- [63] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020.
- [64] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.
- [65] Mirac Suzgun, Nathan Scales, Nathanael Scharli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Annual Meeting of the Association for Computational Linguistics*, 2022.
- [66] Sho Takase, Shun Kiyono, Sosuke Kobayashi, and Jun Suzuki. Spike no more: Stabilizing the pre-training of large language models, 2024.
- [67] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.
- [68] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [69] Qiang Wang, Bei Li, Tong Xiao, Jingbo Zhu, Changliang Li, Derek F. Wong, and Lidia S. Chao. Learning deep transformer models for machine translation. In Anna Korhonen, David Traum, and Lluís Màrquez, editors, *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1810–1822, Florence, Italy, July 2019. Association for Computational Linguistics.
- [70] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max W.F. Ku, Kai Wang, Alex Zhuang, Rongqi "Richard" Fan, Xiang Yue, and Wenhui Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *ArXiv*, abs/2406.01574, 2024.
- [71] Tianwen Wei, Jian Luan, W. Liu, Shuang Dong, and Bin Quan Wang. Cmath: Can your language model pass chinese elementary school math test? *ArXiv*, abs/2306.16636, 2023.
- [72] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.

- [73] Tian Ye, Zicheng Xu, Yuanzhi Li, and Zeyuan Allen-Zhu. Physics of language models: Part 2.1, grade-school math and the hidden reasoning process, 2024.
- [74] Mingjia Yin, Chuhan Wu, Yufei Wang, Hao Wang, Wei Guo, Yasheng Wang, Yong Liu, Ruiming Tang, Defu Lian, and Enhong Chen. Entropy law: The story behind data compression and llm performance. *arXiv preprint arXiv:2407.06645*, 2024.
- [75] Mengxia Yu, De Wang, Qi Shan, Colorado Reed, and Alvin Wan. The super weight in large language models. *ArXiv*, abs/2411.07191, 2024.
- [76] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Annual Meeting of the Association for Computational Linguistics*, 2019.
- [77] Biao Zhang and Rico Sennrich. Root mean square layer normalization, 2019.

A 贡献与致谢

核心贡献者 Yichun Yin, Wenyong Huang, Kaikai Song, Yehui Tang, Xueyu Wu, Wei Guo, Peng Guo, Yaoyuan Wang, Xiaojun Meng, Yasheng Wang, Dong Li, Can Chen, Dandan Tu, Yin Li, Fisher Yu, Ruiming Tang, Yunhe Wang

贡献者 Baojun Wang, Bin Wang, Bo Wang, Boxiao Liu, Changzheng Zhang, Duyu Tang, Fei Mi, Hui Jin, Jiansheng Wei, Jiarui Qin, Jinpeng Li, Jun Zhao, Liqun Deng, Lin Li, Minghui Xu, Naifu Zhang, Nianzu Zheng, Qiang Li, Rongju Ruan, Shengjun Cheng, Tianyu Guo, Wei He, Wei Li, Weiwen Liu, Wulong Liu, Xinyi Dai, Yonghan Dong, Yu Pan, Yue Li, Yufei Wang, Yujun Li, Yunsheng Ni, Zhe Liu, Zhenhe Zhang, Zhicheng Liu